

Содержание

1. Цель работы	3
2. Теоретическое введение	3
2.1. От одного нейрона — к сети	3
2.1.1. Что делает один нейрон	3
2.1.2. Почему одного нейрона мало	4
2.1.3. Многослойный перцептрон (MLP)	4
2.1.4. Соглашение о числе слоёв	5
2.2. Вычислительный граф MLP	5
2.3. Прямой проход (Forward Pass) — пошагово	5
2.4. Функции активации	6
2.4.1. Сигмоида (Sigmoid)	6
2.4.2. Гиперболический тангенс (Tanh)	6
2.4.3. ReLU (Rectified Linear Unit)	7
2.5. Функции ошибки (потерь)	7
2.5.1. Для регрессии — MSE (Mean Squared Error)	8
2.5.2. Для классификации — кросс-энтропия (Cross-Entropy)	8
2.6. Обратное распространение ошибки (Backpropagation)	8
2.6.1. Идея	8
2.6.2. Пошаговое объяснение для сети из двух слоёв	9
2.7. Градиентный спуск и оптимизатор Adam	10
2.7.1. Обычный градиентный спуск (SGD)	10
2.7.2. Adam — адаптивный метод	10
2.8. Ранняя остановка (Early Stopping)	11
2.9. Переобучение (Overfitting) — что это и как его увидеть	11
2.10. Стандартизация данных	12
2.11. Кросс-валидация (K-fold Cross-Validation)	12
2.11.1. Зачем нужна	12
2.11.2. Алгоритм	13
3. Программная реализация	13
3.1. Построение MLP	13
3.2. Обучение модели	14
3.3. Генерация данных	16
3.3.1. Регрессия	16
3.3.2. Классификация	16
3.4. Метрики	17
3.4.1. MSE (регрессия)	17
3.4.2. Ассурасу (классификация)	17

3.4.3. Confusion Matrix (матрица ошибок)	17
4. Часть I. Регрессия на выборках из задания 1	17
4.1. Описание эксперимента	17
4.2. Сводные таблицы Test MSE	18
4.2.1. Полином + равномерный шум	18
4.2.2. $x \sin(2\pi x)$ + нормальный шум	19
4.3. Визуализация лучших моделей	19
4.4. Демонстрация переобучения	19
5. Часть II. Классификация на выборках из задания 2	20
5.1. Описание эксперимента	20
5.2. Визуализация датасетов	21
5.3. Сводные таблицы Test Accuracy	21
5.4. Визуализация лучших моделей	21
6. Оценка минимального размера обучающей выборки	22
6.1. Постановка задачи	22
6.2. Реализация	22
6.3. Результаты	23
7. K-fold кросс-валидация	23
7.1. Постановка задачи	23
7.2. Реализация	23
7.3. Результаты	25
8. Выводы	25
9. Структура проекта	26
10. Репозиторий	26

1. Цель работы

Цель лабораторной работы — реализация многослойного перцептрона (МСП) средствами библиотеки `torch.nn` и исследование его возможностей при решении задач регрессии и классификации.

В работе исследуются:

- три функции активации (Sigmoid, Tanh, ReLU);
- архитектуры с числом линейных слоёв от 2 до 4;
- число нейронов в скрытом слое от 1 до 5;
- регрессия на выборках из задания 1 (полином 3-й степени и осциллирующая функция $x \sin(2\pi x)$);
- классификация на выборках из задания 2 (Circles, XOR, Blobs, Spiral);
- оценка минимального размера обучающей выборки для достижения ассурасы $\geq 90\%$;
- K-fold кросс-валидация для подбора функции активации и числа слоёв.

2. Теоретическое введение

2.1. От одного нейрона — к сети

2.1.1. Что делает один нейрон

Нейрон — это простейший вычислительный элемент. Он получает на вход набор чисел $x_1, x_2, \dots, x_n$, умножает каждое на свой *вес* $w_1, w_2, \dots, w_n$, складывает всё и добавляет *смещение* b :

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = \sum_{j=1}^n w_j x_j + b.$$

Запись в матричном виде: $z = \mathbf{w}^\top \mathbf{x} + b$, где $\mathbf{w} = (w_1, \dots, w_n)^\top$ — вектор-столбец весов, $\mathbf{x} = (x_1, \dots, x_n)^\top$ — вектор-столбец входа.

Затем к z применяется *функция активации* φ , и выход нейрона:

$$a = \varphi(z).$$

Аналогия. Нейрон — это маленький «решатель»: он берёт входные данные, взвешивает каждый фактор по важности (w_j), корректирует порог (b) и через функцию активации выдаёт ответ. Например, при единственном входе x нейрон вычисляет $a = \varphi(wx + b)$ — это изогнутая прямая: если φ — сигмоида, нейрон

вернёт число от 0 до 1 в зависимости от того, насколько x «сдвинут» относительно порога.

2.1.2. Почему одного нейрона мало

Один нейрон без функции активации вычисляет *линейную* функцию входов. Он может разделить данные только прямой (или гиперплоскостью в $\mathbb{R}^n$). Если данные нельзя разделить прямой (например, два вложенных кольца или XOR-задача), один нейрон ошибётся.

Комбинация нескольких нейронов *без* нелинейности тоже не поможет: произведение двух линейных преобразований — линейное преобразование:

$$W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = \underbrace{W^{(2)}W^{(1)}}_{W'}x + \underbrace{W^{(2)}b^{(1)} + b^{(2)}}_{b'}.$$

Отсюда ключевой вывод: без нелинейности сколько бы слоёв мы ни поставили, вся сеть сведётся к одному линейному слою. Именно *функция активации* φ разрывает эту линейность и позволяет сети моделировать сложные зависимости.

2.1.3. Многослойный перцептрон (MLP)

Многослойный перцептрон (MLP — Multilayer Perceptron) — нейронная сеть прямого распространения, состоящая из *входного*, одного или нескольких *скрытых* и *выходного* слоёв. Информация идёт строго в одном направлении: от входа к выходу; обратных связей нет.

Каждый скрытый слой l выполняет два действия:

1. **Линейное преобразование:** $z^{(l)} = W^{(l)}h^{(l-1)} + b^{(l)}$, где $W^{(l)}$ — матрица весов размера (число нейронов слоя $l \times$ число нейронов слоя $l-1$), $b^{(l)}$ — вектор смещений.
2. **Нелинейная активация:** $h^{(l)} = \varphi(z^{(l)})$.

Для входного слоя: $h^{(0)} = x$ — просто сами данные.

Выходной слой — линейный (без активации для регрессии) или с softmax для классификации.

Пример. Рассмотрим сеть с одним скрытым слоем (2 линейных слоя), вход $x \in \mathbb{R}^1$, скрытый слой из 3 нейронов, выход $\hat{y} \in \mathbb{R}^1$:

1. Скрытый слой: $z^{(1)} = W^{(1)}x + b^{(1)}$ — вектор из 3 чисел.
2. Активация: $h^{(1)} = \varphi(z^{(1)})$ — к каждому из 3 чисел применяется φ .
3. Выходной слой: $\hat{y} = W^{(2)}h^{(1)} + b^{(2)}$ — одно число (предсказание).

Здесь $W^{(1)}$ — матрица 3×1 (3 веса), $W^{(2)}$ — матрица 1×3 (ещё 3 веса). Итого 6 весов + 4 смещения = 10 параметров.

2.1.4. Соглашение о числе слоёв

В этой работе «число слоёв» = число **линейных** слоёв (включая выходной).

Линейных слоёв	Скрытых слоёв	Пример (вход $\rightarrow$ выход)
2	1	$x \rightarrow [h^{(1)}] \rightarrow \hat{y}$
3	2	$x \rightarrow [h^{(1)}] \rightarrow [h^{(2)}] \rightarrow \hat{y}$
4	3	$x \rightarrow [h^{(1)}] \rightarrow [h^{(2)}] \rightarrow [h^{(3)}] \rightarrow \hat{y}$

2.2. Вычислительный граф MLP

Для визуального понимания того, как данные «проходят» через сеть, полезен *вычислительный граф*.

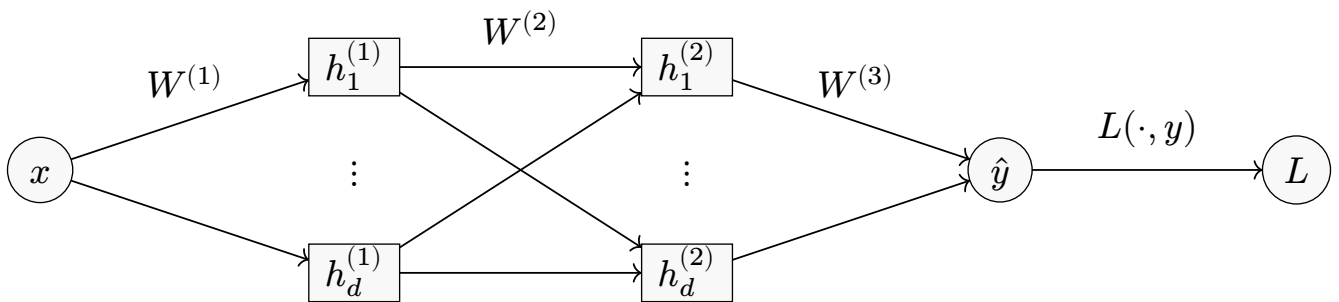


Рис. 1. Вычислительный граф MLP с двумя скрытыми слоями (d нейронов в каждом). L — функция потерь, сравнивающая предсказание $\hat{y}$ с истинным ответом y .

Здесь стрелки означают:

- $x \rightarrow h^{(1)}$: входные данные умножаются на матрицу весов $W^{(1)}$, добавляется смещение, применяется активация;
- $h^{(1)} \rightarrow h^{(2)}$: то же самое, но уже от первого скрытого слоя ко второму;
- $h^{(2)} \rightarrow \hat{y}$: последний линейный слой выдаёт предсказание;
- $\hat{y} \rightarrow L$: предсказание сравнивается с истинным ответом y через функцию потерь.

2.3. Прямой проход (Forward Pass) — пошагово

Прямой проход — это вычисление выхода сети по входу. Рассмотрим конкретный пример: сеть с одним скрытым слоем, 2 нейронами, вход $x = 0.5$, функция активации — сигмоида.

Шаг 1. Линейное преобразование в скрытом слое.

Допустим, $W^{(1)} = \begin{pmatrix} 0.3 \\ -0.8 \end{pmatrix}$, $b^{(1)} = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}$.

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{pmatrix} 0.3 \\ -0.8 \end{pmatrix} \cdot 0.5 + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} 0.25 \\ -0.6 \end{pmatrix}.$$

Шаг 2. Активация.

$$h^{(1)} = \sigma(z^{(1)}) = \begin{pmatrix} \sigma(0.25) \\ \sigma(-0.6) \end{pmatrix} = \begin{pmatrix} 0.562 \\ 0.354 \end{pmatrix}.$$

Шаг 3. Выходной линейный слой.

Допустим, $W^{(2)} = (0.5 \ -0.4)$, $b^{(2)} = 0.1$.

$$\hat{y} = W^{(2)}h^{(1)} + b^{(2)} = 0.5 \cdot 0.562 + (-0.4) \cdot 0.354 + 0.1 = 0.239.$$

Итого: подали $x = 0.5$, получили $\hat{y} = 0.239$. Если правильный ответ $y = 1.0$, потеря (MSE для одного примера) равна $(0.239 - 1.0)^2 = 0.579$.

Задача обучения — подобрать W и b так, чтобы потеря была минимальной на всей выборке.

2.4. Функции активации

Нелинейная функция активации вводится после каждого линейного преобразования *скрытого* слоя. Она определяет, как нейрон «трансформирует» свой линейный выход.

2.4.1. Сигмоида (Sigmoid)

$$\sigma(a) = \frac{1}{1 + e^{-a}}, \quad \sigma(a) \in (0, 1).$$

Что делает: «сжимает» любое число в отрезок $(0, 1)$.

Производная: $\sigma'(a) = \sigma(a)(1 - \sigma(a))$. Максимальное значение производной — 0.25 (при $a = 0$). Это значит, что при каждом прохождении через слой градиент умножается максимум на 0.25.

Проблема затухания градиента: при $|a| \gg 0$ производная стремится к нулю: $\sigma'(5) \approx 0.0066$, $\sigma'(10) \approx 0.000045$. Если в сети 4 скрытых слоя, градиент на первом слое может уменьшиться в $0.25^4 \approx 0.004$ раз — почти ничего не дойдёт, и первые слои практически перестают обучаться.

2.4.2. Гиперболический тангенс (Tanh)

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}, \quad \tanh(a) \in (-1, 1).$$

Что делает: «сжимает» число в $(-1, 1)$, но теперь выход *центрирован в нуле*, что помогает следующему слою: среднее значений близко к 0, и градиенты не имеют систематического смещения.

Производная: $\tanh'(a) = 1 - \tanh^2(a)$. Максимум — 1 (при $a = 0$), что лучше, чем 0.25 у сигмоиды.

Проблема: при $|a| \gg 0$ тоже затухает, но медленнее, чем у сигмоиды. На практике Tanh обычно сходится быстрее Sigmoid.

2.4.3. ReLU (Rectified Linear Unit)

$$\text{ReLU}(a) = \max(0, a) = \begin{cases} a & \text{если } a > 0 \\ 0 & \text{если } a \leq 0 \end{cases}$$

Что делает: оставляет положительные значения как есть, а отрицательные обнуляет.

Производная:

$$\text{ReLU}'(a) = \begin{cases} 1 & \text{если } a > 0 \\ 0 & \text{если } a < 0 \end{cases}$$

При $a > 0$ градиент проходит *без потерь* (множитель 1), поэтому ReLU не подвержена затуханию градиента для положительной области.

Проблема «мёртвых нейронов»: если выход нейрона стал отрицательным для всех обучающих примеров, производная будет всегда 0, и нейрон «умрёт» — перестанет обновляться. Чем больше learning rate, тем выше шанс «убить» нейрон.

Sigmoid	Tanh	ReLU
Выход: $(0, 1)$	Выход: $(-1, 1)$	Выход: $[0, +\infty)$
Гладкая, насыщенная	Центрирована в нуле	Быстрая, без насыщения
$\max \sigma' = 0.25$	$\max \tanh' = 1$	Градиент = 0 или 1
Затухание градиента	Меньше затухание	«Мёртвые нейроны»

Рис. 2. Сравнение трёх функций активации.

2.5. Функции ошибки (потерь)

Функция потерь — это «мера неудовлетворённости» модели: она показывает, насколько предсказания отличаются от правильных ответов. Цель обучения — минимизировать эту функцию.

2.5.1. Для регрессии — MSE (Mean Squared Error)

$$\text{MSE} = E(w) = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2,$$

где $\hat{y}^{(i)} = \tilde{f}(w, x^{(i)})$ — предсказание сети для i -го объекта, $y^{(i)}$ — истинное значение, N — число объектов.

Интуиция: MSE вычисляет средний квадрат ошибки. Квадрат нужен, чтобы: (1) ошибки не компенсировали друг друга; (2) большие ошибки «штрафовались» сильнее, чем маленькие. Если модель ошиблась на 2, вклад — 4, а если на 0.1 — всего 0.01.

2.5.2. Для классификации — кросс-энтропия (Cross-Entropy)

При бинарной классификации с двумя выходными нейронами и softmax:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=0}^1 y_c^{(i)} \log \hat{y}_c^{(i)},$$

где $\hat{y}_c = \text{softmax}(z)_c$ — предсказанная вероятность класса c :

$$\hat{y}_c = \text{softmax}(z)_c = \frac{e^{z_c}}{e^{z_0} + e^{z_1}}.$$

Интуиция: softmax превращает два числа (логита) в вероятности, которые в сумме дают 1. Кросс-энтропия «штрафует» модель за *уверенность в неправильном классе*: если истинный класс — 1, а модель даёт $\hat{y}_1 = 0.01$ (почти уверена, что 0), штраф — $-\log(0.01) \approx 4.6$. Если $\hat{y}_1 = 0.99$ (почти уверена, что 1), штраф — $-\log(0.99) \approx 0.01$.

Почему не MSE для классификации? MSE плохо работает с вероятностями: его ландшафт потерь имеет плоские области (градиент близок к нулю вдали от оптимума), из-за чего обучение замедляется. Кросс-энтропия имеет более крутой градиент вдали от правильного ответа, что ускоряет сходимость.

2.6. Обратное распространение ошибки (Backpropagation)

Обратное распространение — алгоритм вычисления *градиента* функции потерь по каждому весу сети. Это *не* метод оптимизации, а способ эффективно вычислить $\partial \frac{L}{\partial} w$ для всех весов.

2.6.1. Идея

Допустим, мы знаем, как потеря L зависит от выхода последнего слоя. Цепное правило (chain rule) позволяет «протянуть» эту зависимость назад — слой за слоем — до самого первого:

$$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial W^{(l)}}.$$

2.6.2. Пошаговое объяснение для сети из двух слоёв

Рассмотрим простейшую сеть: вход x , один скрытый слой, выход $\hat{y}$, потеря MSE.

Прямой проход:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad h = \varphi(z^{(1)}), \quad \hat{y} = W^{(2)}h + b^{(2)}.$$

Потеря:

$$L = (\hat{y} - y)^2.$$

Обратный проход (вычисляем градиенты):

1. *Градиент потери по выходу:*

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y).$$

2. *Градиент по весам выходного слоя:*

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot h^\top = 2(\hat{y} - y) \cdot h^\top.$$

Это произведение «сигнала ошибки» на входы слоя.

3. *Градиент «проталкивается» обратно через выходной слой:*

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y).$$

$$\frac{\partial L}{\partial h} = (W^{(2)})^\top \delta^{(2)}.$$

4. *Градиент «проталкивается» через функцию активации:*

$$\frac{\partial L}{\partial z^{(1)}} = \frac{\partial L}{\partial h} \odot \varphi'(z^{(1)}).$$

Здесь $\odot$ — поэлементное умножение. Если $\varphi = \sigma$, то $\varphi'(z) = \sigma(z)(1 - \sigma(z))$.

5. *Градиент по весам первого слоя:*

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial z^{(1)}} \cdot x^\top.$$

Общий паттерн: для каждого слоя градиент по весам равен произведению «сигнала ошибки, пришедшего сверху» на «входы этого слоя».

Ключевой момент. Обратное распространение — это *цепное правило*, применённое послойно. PyTorch делает это автоматически: при вызове `loss.backward()` каждый модуль (`nn.Linear`, `nn.Sigmoid` и т.д.) знает свою локальную производную и передаёт градиент предыдущему слою. Нам не нужно писать эти формулы вручную.

2.7. Градиентный спуск и оптимизатор Adam

2.7.1. Обычный градиентный спуск (SGD)

После того как `gradient (backpropagation)` вычислен для каждого веса, обновление весов простейшим методом:

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w_t},$$

где η — *learning rate* (шаг обучения).

Проблема: один и тот же η для всех весов. Если ландшафт потерь «вытянутый» (в одном направлении крутой, в другом — пологий), SGD будет осциллировать вперёд-назад вдоль крутого направления и медленно двигаться вдоль пологого.

2.7.2. Adam — адаптивный метод

Adam (Adaptive Moment Estimation) решает эту проблему, подстраивая *learning rate* для каждого веса индивидуально. Он отслеживает два «момента» градиента:

1-й момент (среднее градиента):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

где $g_t = \frac{\partial L}{\partial w_t}$ — градиент на шаге t , $\beta_1 = 0.9$ (обычно).

Смысл: m_t — «усреднённое направление» градиента за последние шаги. Это сглаживает шум и помогает не «прыгать» из-за отдельных примеров.

2-й момент (среднее квадрата градиента):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

где $\beta_2 = 0.999$ (обычно).

Смысл: v_t — «насколько большим был градиент в среднем». Если градиент стабильно большой, v_t будет большим, и мы уменьшим шаг, чтобы не «перепрыгнуть». Если маленький — увеличим.

Коррекция смещения (поскольку $m_0 = v_0 = 0$, первые оценки занижены):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Обновление весов:

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}},$$

где $\varepsilon = 10^{-8}$ — защита от деления на ноль.

Простыми словами. Adam — это «умный» градиентный спуск, который: (1) запоминает, куда шёл раньше (момент), (2) подстраивает размер шага для каждого веса отдельно. В результате сходится быстрее и стабильнее, чем простой SGD.

2.8. Ранняя остановка (Early Stopping)

При обучении нейросети мы разделяем данные на *обучающую* и *тестовую* (валидационную) части. На каждой эпохе вычисляются потери на обеих частях.

Типичная картина:

- обучающая ошибка монотонно уменьшается;
- тестовая ошибка сначала уменьшается, а потом начинает *расти*.

Момент, когда тестовая ошибка начинает расти — начало *переобучения*: модель запоминает шум в обучающих данных вместо истинной закономерности.

Ранняя остановка — это стратегия: на каждой эпохе мы проверяем, стала ли тестовая ошибка лучше предыдущей наилучшей. Если да — сохраняем состояние модели (веса). В конце обучения загружаем сохранённые «лучшие» веса.

```
if val_loss.item() < best_val:
    best_val = float(val_loss.item())
    best_state = copy.deepcopy(model.state_dict())
```

Результат: модель возвращается в состояние до начала переобучения. Это самый простой и эффективный способ регуляризации.

2.9. Переобучение (Overfitting) — что это и как его увидеть

Переобучение происходит, когда модель *слишком хорошо* подстроилась под обучающие данные, включая их шум и случайные особенности.

Причины:

- **Мало данных.** На 10 точках сеть из 500 параметров легко «выучит» каждую точку отдельно.

- **Слишком сложная модель.** Глубокая сеть с большим числом нейронов имеет огромную «ёмкость» — она может аппроксимировать почти любую функцию, включая шум.
- **Слишком долгое обучение.** Без ранней остановки модель продолжает подстраиваться под шум.

Как распознать:

- train loss падает, а test loss после некоторой эпохи начинает расти;
- модель показывает «дикие» колебания предсказания между точками обучающей выборки (слишком «извилистая» кривая для регрессии).

В эксперименте мы демонстрируем это, обучая сеть с 8 скрытыми слоями на всего 10 точках без ранней остановки.

2.10. Стандартизация данных

Перед обучением признаки (и для регрессии — целевые значения) стандартизируются:

$$\hat{x} = \frac{x - \mu}{\sigma},$$

где μ — среднее, σ — стандартное отклонение, вычисленные *только по обучающей части* (чтобы информация из тестовой части не «утекла» в обучение).

Зачем нужна стандартизация:

1. **Одинаковый масштаб.** Если один признак — в единицах $[0, 1]$, а другой — в $[0, 1000]$, градиент по второму будет доминировать, и обучение станет нестабильным.
2. **Ускорение сходимости.** Стандартизация делает ландшафт потерь более «круглым» (изотропным), и Adam быстрее находит минимум.
3. **Лучшая работа активаций.** Sigmoid и Tanh работают в «чувствительной зоне» при входах порядка $[-2, 2]$. Если входы — порядка 10^3 , нейрон сразу попадает в зону насыщения, и градиент исчезает.

Важно: при предсказании результат нужно *обратно де-стандартизировать*: $y_{\text{pred}} = \hat{y}_{\text{pred}} \cdot \sigma_y + \mu_y$.

2.11. Кросс-валидация (K-fold Cross-Validation)

2.11.1. Зачем нужна

При обычном разбиении train/test результат зависит от того, *какие* объекты попали в тест. Один за разбиением может дать ассурасу 95%, а другое — 85%. Как понять, какая модель действительно лучше?

Кросс-валидация — метод, который усредняет результат по нескольким разбиениям, давая более надёжную оценку.

2.11.2. Алгоритм

Выборка S случайно делится на K равных подмножеств (фолдов) $S_1, S_2, \dots, S_K$.

На каждой итерации $i = 1, \dots, K$:

- **обучающая часть:** $S'_i = \bigcup_{j \neq i} S_j$;
- **тестовая (валидационная) часть:** $T'_i = S_i$.

Кросс-валидация:

1. K раз проводим обучение и тестирование (валидацию) на S'_i, T'_i ($i = 1, \dots, K$);
2. усредняем результаты проверок;
3. выбираем модель с наименьшей ошибкой.

Пример ($K = 5, N = 500$):

- Фолд 1: объекты 1–100 — тест, 101–500 — обучение.
- Фолд 2: объекты 101–200 — тест, 1–100 + 201–500 — обучение.
- ...
- Фолд 5: объекты 401–500 — тест, 1–400 — обучение.

Каждый объект ровно один раз побывал в тестовой части.

3. Программная реализация

3.1. Построение MLP

В PyTorch нейросеть строится как последовательность модулей `nn.Sequential`. Каждый скрытый слой состоит из двух компонентов:

1. `nn.Linear(in_features, out_features)` — линейное преобразование $z = Wx + b$;
2. активация (`nn.Sigmoid()` / `nn.Tanh()` / `nn.ReLU()`) — нелинейное преобразование $h = \varphi(z)$.

Выходной слой — только `nn.Linear` (без активации для регрессии; для классификации `CrossEntropyLoss` сама применяет softmax).

Исходный файл: `mlp.py`

```
ACTIVATIONS = {
    'sigmoid': nn.Sigmoid,
    'tanh':    nn.Tanh,
    'relu':    nn.ReLU,
}
```

```
def build_mlp(input_dim, output_dim, hidden_layers, hidden_dim, activation):
    layers = []
    in_f = input_dim
    act_cls = ACTIVATIONS[activation]

    for _ in range(hidden_layers):
        layers.append(nn.Linear(in_f, hidden_dim))
        layers.append(act_cls())
        in_f = hidden_dim

    layers.append(nn.Linear(in_f, output_dim))
    return nn.Sequential(*layers).to(device)
```

Как это работает пошагово:

1. Словарь `ACTIVATIONS` связывает строковое имя (например, `'relu'`) с классом PyTorch (`nn.ReLU`).
2. В цикле `for _ in range(hidden_layers)` создаётся каждый скрытый слой: сначала линейный (`nn.Linear`), затем активация. Переменная `in_f` отслеживает размерность на «входе» текущего слоя.
3. В конце добавляется выходной `nn.Linear` без активации.
4. `nn.Sequential` собирает все модули в цепочку: при вызове `model(x)` данные последовательно проходят через каждый модуль.

Параметры:

- `input_dim` — размерность входа (1 для регрессии, 2 для классификации);
- `output_dim` — размерность выхода (1 для регрессии, 2 для классификации);
- `hidden_layers` — количество *скрытых* слоёв;
- `hidden_dim` — число нейронов в каждом скрытом слое;
- `activation` — строка: `'sigmoid'`, `'tanh'` или `'relu'`.

3.2. Обучение модели

Функция `train_model` реализует универсальное обучение.

Исходный файл: `mlp.py`

```
def train_model(model, X_train, y_train, X_val, y_val,
                task='regression', epochs=300, lr=0.01):
    loss_fn = nn.MSELoss() if task == 'regression' else nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    history = {'train_loss': [], 'val_loss': []}
    best_state = None
    best_val = float('inf')

    for _ in range(epochs):
        model.train()
```

```

optimizer.zero_grad()
out = model(X_train)
loss = loss_fn(out, y_train)
loss.backward()
optimizer.step()

model.eval()
with torch.no_grad():
    val_out = model(X_val)
    val_loss = loss_fn(val_out, y_val)

history['train_loss'].append(float(loss.item()))
history['val_loss'].append(float(val_loss.item()))

if val_loss.item() < best_val:
    best_val = float(val_loss.item())
    best_state = copy.deepcopy(model.state_dict())

if best_state is not None:
    model.load_state_dict(best_state)
return history

```

Разбор каждого шага:

1. `loss_fn` — выбираем функцию потерь: MSE для регрессии, CrossEntropy для классификации.
2. `optimizer = torch.optim.Adam(...)` — создаём оптимизатор Adam, которому передаём *все* параметры модели и learning rate.
3. В цикле по эпохам:
 - `model.train()` — переводим сеть в режим обучения (включает dropout, batch normalization, если есть);
 - `optimizer.zero_grad()` — обнуляем градиенты, оставшиеся с прошлой эпохи;
 - `out = model(X_train)` — прямой проход: вычисляем предсказания;
 - `loss = loss_fn(out, y_train)` — считаем потерю;
 - `loss.backward()` — обратное распространение: вычисляем градиенты по всем весам;
 - `optimizer.step()` — обновляем веса по формуле Adam;
 - `model.eval()` и `torch.no_grad()` — переводим в режим оценки и отключаем вычисление градиентов (экономим память);
 - сохраняем лучшее состояние, если val loss улучшилось.
4. В конце загружаем лучшие веса обратно в модель.

3.3. Генерация данных

3.3.1. Регрессия

Используются две функции из задания 1:

1. **Случайный полином 3-й степени:** $f(x) = ax^3 + bx^2 + cx + d$ с коэффициентами из $[-3, 3]$.
2. **Осциллирующая функция:** $f(x) = x \sin(2\pi x)$ — функция с возрастающей амплитудой колебаний.

К полиному добавляется *равномерный* шум $\varepsilon \sim U(-\varepsilon_0, \varepsilon_0)$, к осциллирующей — *нормальный* шум $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

Исходный файл: generators.py

```
def random_polynomial(seed=SEED):
    rng = np.random.default_rng(seed)
    a, b, c, d = rng.uniform(-3, 3, 4)
    def f(x):
        return a * x**3 + b * x**2 + c * x + d
    return f, (a, b, c, d)

def trig_function():
    def f(x):
        return x * np.sin(2 * np.pi * x)
    return f
```

3.3.2. Классификация

Используются четыре типа выборок из задания 2:

- **Circles** — два концентрических кольца (нелинейно разделимые);
- **XOR** — четыре облака в углах квадрата (классическая задача, нерешаемая одним нейроном);
- **Blobs** — два разнесённых облака (линейно разделимые);
- **Spiral** — две переплетённые спирали (самая сложная задача).

```
CLS_DATASETS = [
    ('Circles', make_circles),
    ('XOR',     make_xor),
    ('Blobs',   make_blobs),
    ('Spiral',  make_spiral),
]
```


3.4. Метрики

3.4.1. MSE (регрессия)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2.$$

Чем меньше MSE, тем лучше. $\text{MSE} = 0$ означает идеальное предсказание.

3.4.2. Accuracy (классификация)

$$\text{accuracy} = \frac{\text{верно классифицированные}}{\text{все объекты}} = \frac{T_P + T_N}{T_P + F_P + T_N + F_N},$$

где:

- T_P (True Positives) — верно предсказанные как класс 1;
- T_N (True Negatives) — верно предсказанные как класс 0;
- F_P (False Positives) — ошибочно предсказанные как класс 1;
- F_N (False Negatives) — ошибочно предсказанные как класс 0.

3.4.3. Confusion Matrix (матрица ошибок)

$$\begin{pmatrix} T_N & F_P \\ F_N & T_P \end{pmatrix}$$

Каждая ячейка показывает, сколько объектов попало в соответствующую категорию. По диагонали — правильные предсказания, вне диагонали — ошибки.

```
def classification_metrics(y_true, y_pred):
    y_true = np.asarray(y_true).ravel()
    y_pred = np.asarray(y_pred).ravel()
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))
    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))
    acc = (tp + tn) / max(tp + tn + fp + fn, 1)
    return {'accuracy': float(acc), 'confusion_matrix': np.array([[tn, fp], [fn, tp]])}
```

4. Часть I. Регрессия на выборках из задания 1

4.1. Описание эксперимента

Для каждой из двух функций (полином и $x \sin(2\pi x)$) строится MLP с перебором *всех* комбинаций:

- функция активации: `sigmoid`, `tanh`, `relu`;
- число скрытых слоёв: 1, 2, 3 (линейных слоёв: 2, 3, 4);
- число нейронов в скрытом слое: 1, 2, 3, 4, 5.

Параметры обучения:

- $N = 280$ объектов, разбиение 75/25;
- `epochs = 350`, `learning rate = 0.02`;
- оптимизатор Adam, ранняя остановка по `val loss`.

Всего $3 \times 3 \times 5 = 45$ архитектур на каждую функцию — перебор по сетке (grid search).

Что мы ожидаем увидеть:

- сложная функция $x \sin(2\pi x)$ потребует больше нейронов, чем гладкий полином;
- ReLU и Tanh должны работать лучше Sigmoid (меньше проблем с затуханием градиента);
- увеличение числа слоёв не всегда улучшает результат (при малом числе нейронов может навредить).

4.2. Сводные таблицы Test MSE

В таблицах строки — число линейных слоёв (2, 3, 4), столбцы — число нейронов в скрытом слое ($N = 1, \dots, 5$). Значения — Test MSE.

4.2.1. Полином + равномерный шум

Активация	Слоёв	N=1	N=2	N=3	N=4	N=5
Sigmoid	2					
Sigmoid	3					
Sigmoid	4					
Tanh	2					
Tanh	3					
Tanh	4					
ReLU	2					
ReLU	3					
ReLU	4					

Таблица 1. Test MSE для полинома 3-й степени с равномерным шумом. Значения заполняются по результатам запуска.

4.2.2. $x \sin(2\pi x)$ + нормальный шум

Активация	Слоёв	N=1	N=2	N=3	N=4	N=5
Sigmoid	2					
Sigmoid	3					
Sigmoid	4					
Tanh	2					
Tanh	3					
Tanh	4					
ReLU	2					
ReLU	3					
ReLU	4					

Таблица 2. Test MSE для $x \sin(2\pi x)$ с нормальным шумом. Значения заполняются по результатам запуска.

4.3. Визуализация лучших моделей

Для лучшей модели по каждой функции активации строятся:

- график train/test loss по эпохам (показывает процесс обучения: падает ли loss, есть ли расхождение);
- график истинной функции и предсказания сети (показывает, насколько хорошо сеть «уложилась» в данные).

4.4. Демонстрация переобучения

Для наглядной демонстрации переобучения создаётся «идеальный шторм»:

- осциллирующая функция $f(x) = x \sin(2\pi x)$ (сложная форма);
- *маленькая* обучающая подвыборка ($n_{\text{train}} = 10$ из 280);
- *избыточная* сеть (8 скрытых слоёв, 5 нейронов, $\tanh$);
- обучение *без ранней остановки* (5000 эпох).

Что происходит:

- Сеть с ≈ 250 параметрами пытается подстроиться под 10 точек.
- Первые ≈ 200 эпох: и train, и test loss падают.
- Затем: train loss продолжает падать (сеть всё лучше «запоминает» 10 точек), а test loss начинает *расти* (предсказания на новых данных ухудшаются).
- Финальная кривая предсказания «извивается» между обучающими точками, но далеко от истинной функции.

Исходный файл: `block1_main.py` (функция `run_overfit_demo`)

```
def train_no_early_stop(model, X_train, y_train, X_val, y_val,
                        epochs=2500, lr=0.01):
    loss_fn = nn.MSELoss()
```

```

optimizer = torch.optim.Adam(model.parameters(), lr=lr)
history = {'train_loss': [], 'val_loss': []}

for _ in range(epochs):
    model.train()
    optimizer.zero_grad()
    out = model(X_train)
    loss = loss_fn(out, y_train)
    loss.backward()
    optimizer.step()

    model.eval()
    with torch.no_grad():
        val_loss = loss_fn(model(X_val), y_val)

    history['train_loss'].append(float(loss.item()))
    history['val_loss'].append(float(val_loss.item()))
return history

```

Отличие от `train_model`: нет сохранения лучшего состояния и нет ранней остановки. Модель обучается все 5000 эпох «до конца».

5. Часть II. Классификация на выборках из задания 2

5.1. Описание эксперимента

Для каждого из четырёх датасетов (Circles, XOR, Blobs, Spiral) выполняется перебор архитектур аналогично регрессии:

- функция активации: `sigmoid`, `tanh`, `relu`;
- число скрытых слоёв: 1, 2, 3;
- число нейронов в скрытом слое: 1, 2, 3, 4, 5.

Параметры:

- $N = 500$ объектов, разбиение 75/25;
- `epochs = 250`, `learning rate = 0.03`;
- выходной слой — 2 нейрона (по одному на класс), `CrossEntropyLoss`.

Ключевое отличие от регрессии:

- вход — двумерный (x_1, x_2) , а не одномерный;
- выход — два числа (логита), а не одно. `CrossEntropyLoss` внутри себя применяет `softmax` и сравнивает с меткой класса;
- метрика — ассурасу, а не MSE.

Что мы ожидаем:

- Blobs — линейно разделимая задача, даже 1 нейрон справится;
- Circles — нужна хотя бы 1 скрытый слой с несколькими нейронами;
- XOR — классическая задача, нужен минимум 1 скрытый слой;
- Spiral — самая сложная, потребуется глубокая сеть.

5.2. Визуализация датасетов

5.3. Сводные таблицы Test Accuracy

В таблицах строки — число линейных слоёв (2, 3, 4), столбцы — число нейронов ($N = 1, \dots, 5$). Значения — Test accuracy.

Датасет	Активация	N=1	N=2	N=3	N=4	N=5
Circles	Sigmoid					
Circles	Tanh					
Circles	ReLU					
XOR	Sigmoid					
XOR	Tanh					
XOR	ReLU					
Blobs	Sigmoid					
Blobs	Tanh					
Blobs	ReLU					
Spiral	Sigmoid					
Spiral	Tanh					
Spiral	ReLU					

Таблица 3. Test accuracy для классификации (при 4 линейных слоях). Значения заполняются по результатам запуска.

5.4. Визуализация лучших моделей

Для лучшей модели строятся:

- **график train/test loss** — показывает процесс обучения;
- **граница решений (decision boundary)** — раскрашивает плоскость двумя цветами по предсказанным классам. Чем более сложная граница, тем более «мощную» модель пришлось обучить;
- **confusion matrix** — показывает распределение ошибок.

6. Оценка минимального размера обучающей выборки

6.1. Постановка задачи

Вопрос: сколько объектов нужно для обучения, чтобы классификатор достигал ассигасу $\geq 90\%$?

Ответ зависит от сложности задачи: для простых (Blobs) может хватить нескольких десятков, для сложных (Spiral) — нескольких сотен.

Фиксированные параметры модели:

- функция активации: `tanh`;
- число линейных слоёв: 4;
- число нейронов: 5;
- `epochs = 250`, `lr = 0.03`.

Доля обучающей части варьируется от 10% до 95% с шагом 5%. Для каждой доли обучаем модель и проверяем ассигасу на тестовой части. Минимальная доля, при которой ассигасу $\geq 90\%$ — искомый результат.

6.2. Реализация

Исходный файл: `block2_main.py` (функция `estimate_min_train_size`)

```
def estimate_min_train_size(datasets, activation='tanh', linear_layers=4,
                            hidden_dim=5, target_acc=0.90,
                            epochs=CLS_EPOCHS, lr=CLS_LR):
    fracs = np.arange(0.10, 0.96, 0.05)
    summary = []

    for ds in datasets:
        name = ds['name']
        X, y = ds['data']
        best_frac = None
        for frac in fracs:
            test_ratio = 1.0 - float(frac)
            X_tr, X_te, y_tr, y_te = train_test_split(X, y,
                                                        test_ratio=test_ratio, seed=SEED)
            X_tr_s, X_te_s, _, _ = standardize(X_tr, X_te)

            model = build_mlp(2, 2, linear_layers - 1, hidden_dim, activation)
            train_model(model, ...)

            pred = predict_classes(model, to_tensor(X_te_s))
            acc = classification_metrics(y_te, pred)['accuracy']

            if best_frac is None and acc >= target_acc:
```

```

        best_frac = float(frac)

    summary.append({
        'dataset': name,
        'min_frac': best_frac,
        'min_train_size': int(round(best_frac * len(X)))
                           if best_frac else None,
    })
return summary

```

Логика: для каждого датасета перебираем доли от 10% до 95%. Как только ассигасу впервые превысила 90% — запоминаем эту долю.

6.3. Результаты

Датасет	Активация	Слоёв	Нейронов	Min frac	Min N
Circles	tanh	4	5		
XOR	tanh	4	5		
Blobs	tanh	4	5		
Spiral	tanh	4	5		

Таблица 4. Минимальный размер обучающей выборки для ассигасу $\geq 90\%$. Значения заполняются по результатам запуска.

7. K-fold кросс-валидация

7.1. Постановка задачи

Для каждого датасета из задания 2 определить наилучшие значения:

1. функции активации (sigmoid / tanh / ReLU);
2. числа линейных слоёв (2, 3 или 4).

Число нейронов фиксировано: $d = 5$.

Используется 5-fold кросс-валидация: каждая из $3 \times 3 = 9$ комбинаций (активация $\times$ слои) оценивается на 5 разбиениях.

7.2. Реализация

Исходный файл: `block2_main.py` (функция `cv_grid_search`)

```

def kfold_indices(n, k=5, seed=SEED):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(n)
    return np.array_split(idx, k)

```

```

def cv_grid_search(X, y, activations=('sigmoid', 'tanh', 'relu'),
                  layers_grid=(2, 3, 4), hidden_dim=5,
                  k=CV_K, epochs=CLS_EPOCHS, lr=CLS_LR):
    folds = kfold_indices(len(X), k=k)
    rows = []
    best_row = None

    for act in activations:
        for nl in layers_grid:
            fold_accs = []
            for fi in range(k):
                val_idx = folds[fi]
                tr_idx = np.concatenate([folds[j] for j in range(k) if j != fi])

                X_tr, X_val = X[tr_idx], X[val_idx]
                y_tr, y_val = y[tr_idx], y[val_idx]
                X_tr_s, X_val_s, _, _ = standardize(X_tr, X_val)

                model = build_mlp(2, 2, nl - 1, hidden_dim, act)
                train_model(model, ...)

                pred = predict_classes(model, to_tensor(X_val_s))
                fold_accs.append(
                    classification_metrics(y_val, pred)['accuracy'])

            row = {
                'activation': act, 'linear_layers': nl,
                'cv_mean_acc': float(np.mean(fold_accs)),
                'cv_std_acc': float(np.std(fold_accs, ddof=1)),
            }
            rows.append(row)
            if best_row is None or row['cv_mean_acc'] > best_row['cv_mean_acc']:
                best_row = row

    return rows, best_row

```

Разбор:

1. `kfold_indices` случайно перемешивает индексы и делит на k частей.
2. Тройной цикл: по активациям, по числу слоёв, по фолдам.
3. На каждом фолде: обучение с нуля на $k - 1$ частях, тест на 1 оставшейся.
4. `ddof=1` в `np.std` — несмещённая оценка дисперсии (деление на $k - 1$, а не k).

7.3. Результаты

Датасет	Лучшая активация	Слоёв	Mean acc	Std acc
Circles				
XOR				
Blobs				
Spiral				

Таблица 5. Лучшие результаты 5-fold кросс-валидации. Значения заполняются по результатам запуска.

8. Выводы

По результатам экспериментов:

1. Многослойный перцептрон, реализованный средствами `torch.nn`, успешно решает как задачи регрессии, так и бинарной классификации.
2. Для регрессии MLP аппроксимирует полином 3-й степени и осциллирующую функцию $x \sin(2\pi x)$. Качество аппроксимации зависит от числа нейронов и глубины сети.
3. Функция активации влияет на скорость обучения и качество:
 - `tanh` и `relu` обычно дают лучшие результаты;
 - `sigmoid` медленнее сходится из-за затухания градиента.
4. Для простых задач классификации (Blobs) высокая точность достигается даже при минимальной архитектуре (2 слоя, 1 нейрон).
5. Для сложных задач (Spiral) критически важны глубина сети и число нейронов. Функция активации ReLU и Tanh показывают лучшие результаты.
6. Переобучение наглядно проявляется при малом размере обучающей выборки ($n = 10$) и избыточной сети: train loss падает, а test loss начинает расти. Ранняя остановка позволяет вернуться к лучшему состоянию модели.
7. Кросс-валидация позволяет надёжно выбрать оптимальную архитектуру модели, усредняя результаты по нескольким разбиениям. Она снижает зависимость от конкретного train/test разбиения.
8. Минимальный размер обучающей выборки для $\text{accuracy} \geq 90\%$ зависит от сложности геометрии данных: для Blobs достаточно малого числа объектов, для Spiral требуется значительно больше.

9. Структура проекта

```
SMGMO/  
├─ lab3/  
│   ├── main.typ          # отчёт (Typst)  
│   ├── config.py         # параметры экспериментов (через env-переменные)  
│   ├── generators.py     # генерация данных (регрессия + классификация)  
│   ├── mlp.py            # MLP: построение, обучение, предсказание, метрики  
│   ├── block1_main.py    # Блок 1: регрессия + переобучение  
│   ├── block2_main.py    # Блок 2: классификация + min N + кросс-валидация  
│   └─ gui_launcher.py    # GUI-лаунчер (Tkinter)
```

10. Репозиторий

Исходный код проекта доступен на GitHub: <https://github.com/Q1zin/SMGMO.git>